

Bags, Vectors & Transformers

Classic Text Analysis · Day 1, Part 3

Denise J. Roth, PhD Candidate

Strategic Communication Group · Wageningen University & Research

What Do We Mean by “Classic” Text Analysis?

By **classic** computational text analysis, we mean the family of methods that treat text as **counts of words**. The central idea is simple but powerful: if we can turn documents into **numbers**, we can compare them, measure them, and analyze them statistically.

What it includes

- **Word frequencies**
- **Dictionaries** and lexicons
- **TF-IDF** weighting
- **Topic models** (e.g. LDA)

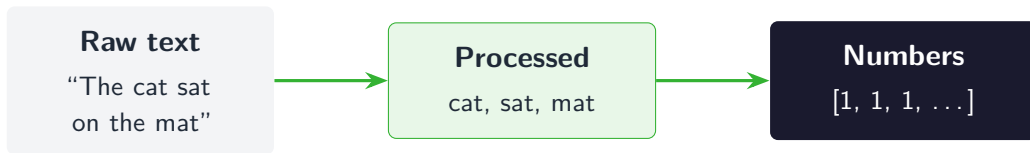
The key simplification

- Word **order** is ignored
- A document becomes a “**bag of words**”
- Only **counts** matter
- Surprisingly effective despite this

A Little History

- 🕒 **1940s–50s:** early word-counting in the humanities — concordances, authorship studies (who *really* wrote this text?)
- 🕒 **1960s: content analysis** formalized in the social sciences; the General Inquirer, one of the first computer-assisted text systems
- 🕒 **1970s–80s: information retrieval** matures — TF-IDF and the vector space model, driven by the need to search documents
- 🕒 **2000s: topic modeling** (LDA, 2003) makes large-scale thematic analysis accessible
- 🕒 **2010s onward:** the field shifts toward **embeddings** and **neural methods** — but the classic tools remain everywhere

The Core Idea: Text as Data



The whole enterprise rests on one move: **transforming text into a numerical representation** a computer can work with. Everything else — counting, weighting, modeling — builds on this. The rest of this session is about the vocabulary you need to understand that move.

Prerequisite Concepts (1): Corpus & Documents

Corpus

The **full collection** of texts you are analyzing. Your dataset. Could be 10 interviews or 10 million tweets.

Document

A **single unit** within the corpus. What counts as one is your choice: a tweet, an article, a paragraph, a response.

Deciding what your “document” is matters — it shapes everything downstream. One survey response? One sentence? One respondent’s full answer set? There is no universally right answer, only what fits your question.

Prerequisite Concepts (2): Tokens, Types, Vocabulary

- **Token:** a single occurrence of a word in a text. “the cat sat on the mat” has **6 tokens**.
- **Type:** a distinct word. That same sentence has **5 types** (“the” appears twice but counts once).
- **Vocabulary:** the **set of all types** across your whole corpus — the columns of the matrix we are about to build.

Why it matters: the size of your vocabulary determines how “wide” your data is. Real corpora can have tens of thousands of types, which is why **preprocessing** (next) is so important.

Prerequisite Concepts (3): The Preprocessing Mindset

Before counting, we usually **clean and normalize** text. Each choice is a **modeling decision**, not just housekeeping — it changes what your numbers mean.

Common steps

- **Lowercasing** (“The” = “the”?)
- Removing **punctuation**
- Removing **stopwords** (the, of, and)
- **Stemming** / **lemmatization**

But be careful

- “not good” vs “good” — stopwords can carry meaning
- Stemming can over-merge words
- Choices depend on your **research question**

Every one of these steps is language-specific. Stopword lists, tokenizers, and lemmatizers all differ by language — an English pipeline will not work on Dutch or German out of the box, and compound words (*gezondheidszorgbeleid*) add a further wrinkle.

The Document-Term Matrix (DTM)

The **document-term matrix** is where it all comes together. Rows are **documents**, columns are **terms** (your vocabulary), and each cell holds a **count**: how often that term appears in that document.

This single structure is the **input** to almost every classic method: word frequencies, TF-IDF, topic models, and even many machine learning classifiers. Once your text is a DTM, it is just a **matrix of numbers** — and decades of statistical tools become available.

Let's build one by hand.

Worked Example: Building a DTM

Three short “documents”:

D1: the cat sat **D2:** the dog sat **D3:** the cat ran

	the	cat	sat	dog	ran
D1	1	1	1	0	0
D2	1	0	1	1	0
D3	1	1	0	0	1

Each **row** is a document as a vector of counts. Each **column** is a term across all documents. That is the entire idea — and everything in the coming sessions builds on this simple table.

A Problem with Raw Counts

Look again at the DTM. The word “**the**” appears in **every** document — so it is useless for telling them apart. Raw counts give **common words too much weight** and **distinctive words too little**.

The issue

- “the”, “of”, “and” appear **everywhere**
- High counts, but **low information**
- They drown out the words that actually **distinguish** documents

The idea

A word is **informative** about a document if it is

- **frequent** in that document, but
- **rare** across the corpus as a whole

*This is exactly what **TF-IDF** captures.*

TF-IDF: Term Frequency $\times$ Inverse Document Frequency

TF-IDF reweights each cell of the DTM by multiplying two quantities:

TF — Term Frequency

How often a term appears **in this document**.

Higher = more important **to that document**.

IDF — Inverse Document Frequency

How **rare** the term is **across the corpus**.

Appears everywhere = low. Appears rarely = high.

The result: words that are frequent in a document *but* rare elsewhere get the **highest scores**. “the” gets crushed to near zero; a distinctive word like “cat” stands out. Still **bag-of-words** — just **smarter weighting**.

Before We Start Coding

You now have the vocabulary you need. Make sure these feel clear:

Concepts

- **Corpus** and **document**
- **Token**, **type**, **vocabulary**
- **Preprocessing** as a decision
- The **document-term matrix**

The big picture

- Text becomes **numbers**
- Word **order is dropped**
- Counts open up **statistics**
- Simple, but a real workhorse

Next: we build these matrices in Python and put them to work.